

Lista de Exercícios — Introdução à Linguagem Kotlin e POO

Instruções de Entrega: Os alunos devem desenvolver o código-fonte em Kotlin para cada um dos problemas. A solução final deve ser compactada e submetida no **Google Classroom** em um arquivo **.zip** contendo os arquivos ou pastas organizados como *exercicio1*, *exercicio2*, *exercicio3*, *exercicio4*, *exercicio5* e *exercicio6*.

“Boa atividade! Att. Me. Prof. Sergio Souza Novak.”

Questão 1 — Cálculo de Média Aritmética e Situação Acadêmica (exercicio1)

Escreva um programa em Kotlin que receba três notas de um aluno (valores do tipo Double), calcule a média aritmética simples e determine a situação acadêmica do estudante segundo os seguintes critérios:

- Média ≥ 7.0 : "Aprovado"
- $5.0 \leq \text{Média} < 7.0$: "Em Recuperação"
- Média < 5.0 : "Reprovado"

Requisito: Implemente uma função ‘calcularMedia(n1: Double, n2: Double, n3: Double): Double’ e utilize *string templates* para formatar a saída com duas casas decimais.

Exemplo de Entrada e Saída Esperada (Console)

Entrada no código / teclado: Nota 1 = 8.5, Nota 2 = 6.0, Nota 3 = 7.5

Saída de Console:

Notas digitadas: 8.5, 6.0, 7.5

Média do aluno: 7.33

Situação: Aprovado

Questão 2 — Analisador e Formatador de Nomes de Usuários (exercicio2)

Em aplicativos móveis, a higienização e validação de strings digitadas pelo usuário são essenciais. Crie um programa em Kotlin que receba um nome completo digitado (exemplo: "sergio souza novak ") e realize as seguintes operações de manipulação de String:

- a) Remova espaços desnecessários no início e no fim da string.

- b) Converta o nome para letras maiúsculas.
- c) Conte a quantidade de vogais (a, e, i, o, u) presentes no nome.
- d) Verifique se o nome contém o sobrenome "Novak" (ignorando diferenças entre maiúsculas e minúsculas).
- e) Gere um nome de exibição no formato: "PrimeiroNome SOBRENOME" (exemplo: "Sergio NOVAK").

Exemplo de Entrada e Saída Esperada (Console)

Entrada: "sergio souza novak "

Saída de Console:

Nome higienizado: "sergio souza novak"

Nome em maiúsculas: SERGIO SOUZA NOVAK

Quantidade de vogais: 8

Contém sobrenome 'Novak'? sim

Nome de Exibição: Sergio NOVAK

Questão 3 — Busca Binária em Coleção de Dados Ordenada (exercicio3)

A busca binária é um algoritmo eficiente de busca com complexidade $O(\log n)$, que exige que a lista esteja previamente ordenada.

Implemente em Kotlin uma função 'buscaBinaria(array: IntArray, chave: Int): Int' que retorna o índice onde a 'chave' foi encontrada ou '-1' caso o valor não pertença ao array.

- Crie uma lista/array de inteiros contendo pelo menos 10 elementos ordenados.
- Demonstre o teste da sua função buscando por um elemento existente e por um elemento inexistente.
- Imprima no console cada passo do algoritmo (valores de `inicio`, `fim` e `meio`).

Exemplo de Entrada e Saída Esperada (Console)

Array de Entrada: [5, 12, 18, 23, 34, 45, 56, 67, 78, 89]

Teste 1 — Buscar chave = 56:

Passo 1: inicio=0, fim=9, meio=4 (valor 34) -> buscar à direita

Passo 2: inicio=5, fim=9, meio=7 (valor 67) -> buscar à esquerda

Passo 3: inicio=5, fim=6, meio=5 (valor 45) -> buscar à direita

Passo 4: inicio=6, fim=6, meio=6 (valor 56) -> ENCONTRADO!

Saída: Chave 56 encontrada no índice 6.

Teste 2 — Buscar chave = 99:

Saída: Chave 99 não encontrada no array (retorno -1).

Questão 4 — Operação de Rotação em Matrizes de Pontos 2D (exercicio4)

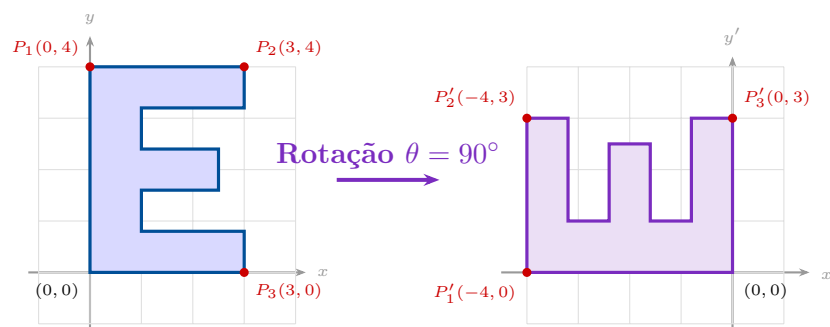
No desenvolvimento de jogos e interfaces gráficas móveis, a rotação de coordenadas no plano cartesiano é uma operação vetorial e matricial fundamental. Dado um ponto (x, y) , a rotação por um ângulo θ (em radianos) no sentido anti-horário em relação à origem $(0, 0)$ é dada pela multiplicação matricial:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

que resulta nas equações: $x' = x \cos \theta - y \sin \theta$ e $y' = x \sin \theta + y \cos \theta$.

Tarefas:

- Crie uma classe em Kotlin chamada 'Ponto2D' com os atributos 'x: Double' e 'y: Double'.
- Adicione um método 'rotacionar(anguloGraus: Double): Ponto2D' que retorne um **novο** 'Ponto2D' com as coordenadas rotacionadas. Utilize a classe 'kotlin.math' (cos, sin, toRadians).
- Desenvolva uma função que receba uma **matriz/array de pontos** (ex: os vértices de um triângulo ou polígono) e aplique a rotação em todos os pontos da matriz.



a) Polígono 'E' Original ($\theta = 0^\circ$) b) Polígono 'E' Rotacionado ($\theta = 90^\circ$)

Multiplicação Matricial para Rotação ($\theta = 90^\circ$):

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(90^\circ) & -\sin(90^\circ) \\ \sin(90^\circ) & \cos(90^\circ) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -y \\ x \end{bmatrix}$$

Exemplo de multiplicação das coordenadas dos pontos vértices da matriz do polígono:

$$P_1 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \begin{bmatrix} -4 \\ 0 \end{bmatrix}, \quad P_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} -4 \\ 3 \end{bmatrix}, \quad P_3 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

Ponto2D
+ x: Double
+ y: Double
+ rotacionar(anguloGraus: Double): Ponto2D
+ toString(): String

Exemplo de Entrada e Saída Esperada (Console)

Pontos de Entrada (Matriz/Array): [Ponto2D(0.0, 4.0), Ponto2D(3.0, 4.0), Ponto2D(3.0, 0.0)]

Ângulo de Rotação: 90.0°

Saída de Console:

--- Pontos Originais ---

P1: (0.00, 4.00)

P2: (3.00, 4.00)

P3: (3.00, 0.00)

--- Pontos Após Rotação de 90.0° ---

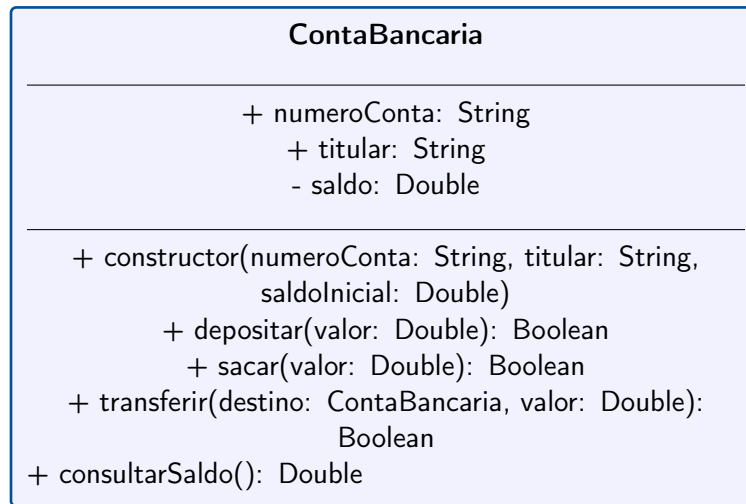
P1': (-4.00, 0.00)

P2': (-4.00, 3.00)

P3': (0.00, 3.00)

Questão 5 — Sistema de Conta Bancária com Operações Financeiras (exercicio5)

Modele e implemente em Kotlin um sistema de conta bancária orientada a objetos com base no diagrama de classe UML a seguir:



Regras de Negócio:

- O atributo `saldo` deve ser estritamente privado (`private`).
- O método `depositar(valor)` só deve aceitar valores estritamente positivos (> 0).
- O método `sacar(valor)` deve verificar se há saldo suficiente para realizar a operação.
- O método `transferir(destino, valor)` deve debitar da conta de origem e creditar na conta de `destino` apenas se o saque de origem for bem-sucedido.
- No `main()`, crie duas contas, realize depósitos, saques e uma transferência entre elas, exibindo os saldos finais.

Exemplo de Entrada e Saída Esperada (Console)

Operações no main(): Criar `c1("101-X", "Sergio", 1000.0)` e `c2("202-Y", "Ana", 500.0)`.

Depositar R\$ 200 em `c1` | Sacar R\$ 150 de `c2` | Transferir R\$ 300 de `c1` para `c2`.

Saída de Console:

[Depósito] R\$ 200.0 depositado na conta 101-X (Sergio). Saldo: R\$ 1200.0

[Saque] R\$ 150.0 sacado da conta 202-Y (Ana). Saldo: R\$ 350.0

[Transferência] R\$ 300.0 transferido de 101-X para 202-Y.

--- Saldo Final ---

Conta 101-X (Sergio): R\$ 900.0

Conta 202-Y (Ana): R\$ 650.0

Questão 6 — Avaliação de Aptidão Textural do Solo para Cultivo de Arroz Irrigado (exercício6)

Texto de Apoio



Figura 6.1: Cultivo de arroz com lâmina de irrigação.



Figura 6.2: Classes de textura do solo.

A textura do solo é uma das propriedades físicas mais importantes para o planejamento e manejo de bacias de irrigação, especialmente em sistemas de arroz irrigado por lâmina d'água (arroz de várzea). Ela é definida pela proporção relativa das frações de argila, areia e silte presentes no solo e determina características essenciais como capacidade de retenção de água, velocidade de infiltração, porosidade e disponibilidade de nutrientes para as plantas.

No cultivo do arroz de várzea, a manutenção de uma lâmina de água constante sobre o solo é fundamental para o desenvolvimento da cultura. Para isso, solos de textura argilosa ou franco-argilosa são considerados ideais, pois apresentam alta retenção hídrica e baixa taxa de infiltração, o que reduz perdas de água por percolação profunda e facilita o controle da irrigação.

Além disso, solos com maior teor de argila possuem elevada microporosidade e maior capacidade de troca catiônica (CTC), o que favorece a retenção de nutrientes e a eficiência do uso da água pela planta. No entanto, esses solos também exigem manejo cuidadoso para evitar compactação excessiva e garantir adequada aeração e desenvolvimento radicular.

Compreender a textura do solo e suas implicações hidrológicas é, portanto, etapa indispensável para o dimensionamento correto de bacias de irrigação, a definição de lâminas de água adequadas e a adoção de práticas de manejo que maximizem a produtividade do arroz irrigado de forma sustentável.

Referência Bibliográfica:

MASCARENHAS, I. S.; GONÇALVES, G. de M. O.; CAETANO, P. H. P.; SANTOS, A. B. dos; MADARI, B. E.; CORRECHEL, V.; SILVA, M. A. S. da. *Características físicas de solos de várzea e produtividade do arroz irrigado por inundação*. Brasília, DF: Empresa Brasileira de Pesquisa Agropecuária (Embrapa), 2016. Resumo em anais e proceedings. Acesso em: 31 jul. 2026.

Do ponto de vista quantitativo, a determinação da textura do solo deve obedecer ao princípio de conservação de massa das frações granulométricas. Assim, a soma das porcentagens de argila, areia e silte deve satisfazer a seguinte restrição:

$$\text{Argila (\%)} + \text{Areia (\%)} + \text{Silte (\%)} = 100\% \quad (1)$$

Essa condição é fundamental para a correta classificação textural do solo por meio do triângulo textural (USDA ou sistema brasileiro), garantindo que as proporções estejam coerentes e que a classe textural identificada represente adequadamente as propriedades físico-hídricas do solo. Com base nas implicações agrônômicas da textura para o arroz irrigado, pode-se definir um **Índice de Aptidão (Fitness)** do solo para essa cultura. Esse índice empírico pondera positivamente as frações de argila e silte (benéficas para retenção hídrica) e negativamente a fração de areia (associada à maior infiltração e menor retenção):

$$\text{Fitness} = (1.5 \times \text{Argila}) + (0.8 \times \text{Silte}) - (0.5 \times \text{Areia}) \quad (2)$$

em que *Argila*, *Silte* e *Areia* são expressos em porcentagem (%) e satisfazem a Equação 1.

Quanto maior o valor de *Fitness*, maior a aptidão textural do solo para o cultivo de arroz irrigado por inundação. Solos argilosos e franco-argilosos, típicos de várzeas, tendem a apresentar valores elevados desse índice, refletindo sua adequação para a manutenção da lâmina d'água e eficiência no uso de nutrientes.

A combinação da restrição granulométrica (Eq. 1) com o índice de aptidão (Eq. 2) oferece uma base quantitativa para o planejamento de bacias de irrigação, seleção de áreas para expansão do arroz irrigado e avaliação de estratégias de manejo que maximizem a produtividade de forma sustentável.

Considere a seguinte classificação de aptidão para a cultura do arroz irrigado:

- $Fitness \geq 60.0$: Excelente Aptidão (Alta retenção hídrica)
- $40.0 \leq Fitness < 60.0$: Aptidão Moderada (Requer maior manejo de irrigação)
- $Fitness < 40.0$: Inadequado (Solo excessivamente permeável/arenoso)

Solo
+ identificacao: String + argila: Double + areia: Double + silte: Double
+ constructor(identificacao: String, argila: Double, areia: Double, silte: Double) + validarTextura(): Boolean + calcularFitness(): Double + obterClassificacao(): String + compararAptidao(outroSolo: Solo): Solo + exibirRelatorio(): Unit

Especificações de Implementação em Kotlin:

- Implemente o construtor da classe 'Solo'. No bloco 'init', valide se $argila + areia + silte = 100.0$ (tolere pequenas diferenças de arredondamento de ± 0.1). Caso inválido, exiba um aviso.
- Implemente o método 'calcularFitness(): Double' com a fórmula fornecida.
- Implemente o método 'obterClassificacao(): String' que retorna o texto da aptidão.
- Método de Comparação: Implemente 'compararAptidao(outroSolo: Solo): Solo', que compara o **fitness** do solo atual (**this**) com o de 'outroSolo' recebido como parâmetro e retorna o objeto 'Solo' que possui a melhor aptidão (maior fitness) para o cultivo de arroz irrigado.
- Na função 'main()', instancie dois solos (ex: *Lote A: 50% Argila, 20% Areia, 30% Silte* e *Lote B: 15% Argila, 70% Areia, 15% Silte*), exiba os relatórios de cada um e utilize o método 'compararAptidao' para indicar qual solo é mais recomendado.

Exemplo de Entrada e Saída Esperada (Console)**Objetos Instanciados:**

```
soloA = Solo("Lote A", argila=50.0, areia=20.0, silte=30.0)
```

```
soloB = Solo("Lote B", argila=15.0, areia=70.0, silte=15.0)
```

Saída de Console:

```
--- Relatório de Solo: Lote A ---
```

```
Composição: 50.0% Argila, 20.0% Areia, 30.0% Silte
```

```
Índice de Fitness: 89.0
```

```
Classificação: Excelente Aptidão (Alta retenção hídrica)
```

```
--- Relatório de Solo: Lote B ---
```

```
Composição: 15.0% Argila, 70.0% Areia, 15.0% Silte
```

```
Índice de Fitness: -0.5
```

```
Classificação: Inadequado (Solo excessivamente permeável/arenoso)
```

```
=====
```

```
Resultado da Comparação de Aptidão:
```

```
O solo recomendado para cultivo de arroz irrigado é: Lote A (Fitness: 89.0)
```

```
=====
```